

✓ **L02- Deploying an AI Model on a Simulated Edge Device

Options 2**

Part 1: Setting Up A Lab Environment ** In Colab**

Step 1: Installing Python And TensorFlow

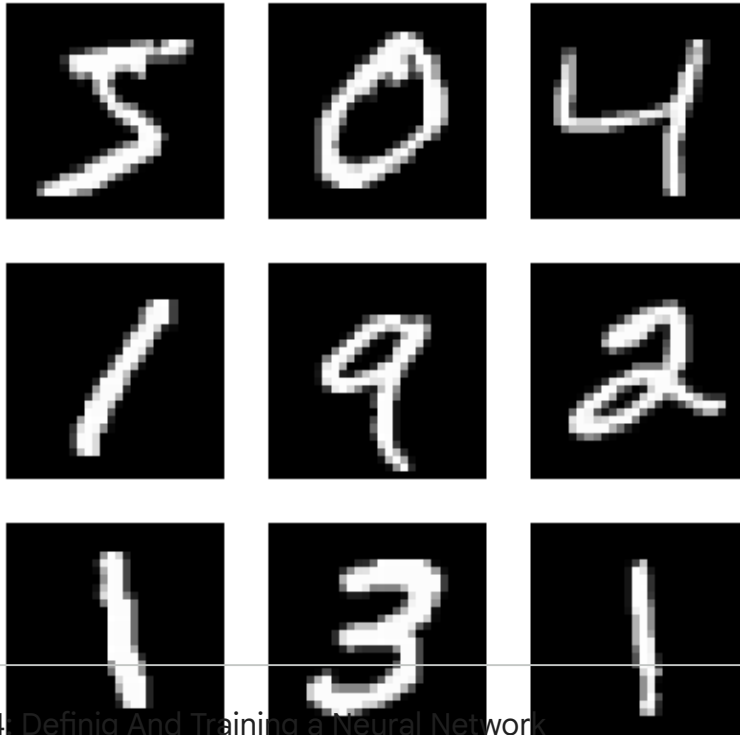
```
1 !python --version
2 !pip show tensorflow
```

```
Python 3.12.12
Name: tensorflow
Version: 2.19.0
Summary: TensorFlow is an open source machine learning framework for everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: /usr/local/lib/python3.12/dist-packages
Requires: absl-py, astunparse, flatbuffers, gast, google-pasta, grpcio, h5py, keras,
Required-by: dopamine_rl, tensorflow-text, tensorflow_decision_forests, tf_keras
```

Part 2: Creating And traing An AI Model

Step 2: Loading MNIST Dataset

```
1 import tensorflow as tf
2 from tensorflow.keras.datasets import mnist
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Load MNIST dataset
7 (x_train, y_train), (x_test, y_test) = mnist.load_data()
8
9 # Normalize data (scale pixel values between 0 and 1)
10 x_train, x_test = x_train / 255.0, x_test / 255.0
11
12 # Show sample images
13 plt.figure(figsize=(5,5))
14 for i in range(9):
15     plt.subplot(3,3,i+1)
16     plt.imshow(x_train[i], cmap="gray")
17     plt.axis('off')
18 plt.show()
```



Step 4: Defining And Training a Neural Network

a. Creation of a Single Feedforward neural network using Keras.

```

1 model = tf.keras.models.Sequential([
2     tf.keras.layers.Flatten(input_shape=(28, 28)),
3     tf.keras.layers.Dense(128, activation="relu"),
4     tf.keras.layers.Dense(10, activation="softmax")
5 ])
6
7 model.compile(
8     optimizer="adam",
9     loss="sparse_categorical_crossentropy",
10    metrics=["accuracy"]
11 )
12
13 history = model.fit(
14     x_train, y_train,
15     epochs=5,
16     validation_data=(x_test, y_test)
17 )
18

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: Use
super().__init__(**kwargs)

```

Epoch 1/5

1875/1875 ————— **10s** 5ms/step - accuracy: 0.8766 - loss: 0.4286 - val_

Epoch 2/5

1875/1875 ————— **9s** 5ms/step - accuracy: 0.9629 - loss: 0.1245 - val_a

Epoch 3/5

1875/1875 ————— **8s** 4ms/step - accuracy: 0.9740 - loss: 0.0853 - val_a

Epoch 4/5

1875/1875 ————— **10s** 5ms/step - accuracy: 0.9814 - loss: 0.0616 - val_

Epoch 5/5

1875/1875 ————— **9s** 5ms/step - accuracy: 0.9858 - loss: 0.0441 - val_a

This Trainig Learned Correctly and did not have any overfitting issues.

Step 5: Converting And Saving the Model as .tflite

```
1 model.save("mnist_model.h5")
2 print("Model saved as mnist_model.h5")
3
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.s...
Model saved as mnist_model.h5

Checking if the Model has been saved

```
1 model.save("mnist_model.keras")
2 !ls /content
3
4
5 # Write model has been saved succefully
6
```

mnist_model.h5 mnist_model.keras sample_data

Step 6: Loading the TensorFlow Lite model (Interpreter)

```
1 import tensorflow as tf
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Convert the Keras model to TensorFlow Lite format
6 converter = tf.lite.TFLiteConverter.from_keras_model(model)
7 tflite_model = converter.convert()
8
9 # Save the converted model
10 with open('mnist_model.tflite', 'wb') as f:
11     f.write(tflite_model)
12
13 print("Model converted and saved as mnist_model.tflite")
14
15 # Load TensorFlow Lite model
16 interpreter = tf.lite.Interpreter(model_path="mnist_model.tflite")
17 interpreter.allocate_tensors()
18
19 # Get input and output tensor details
20 input_details = interpreter.get_input_details()
21 output_details = interpreter.get_output_details()
22
23 print("Input Details:", input_details)
24 print("Output Details:", output_details)
25
```

ifact at '/tmp/tmpw4x98jv6'. The following endpoints are available:

```

t 'serve'
(POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.float32, name='keras_tensor')
TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)

37957968: TensorSpec(shape=(), dtype=tf.resource, name=None)
37958928: TensorSpec(shape=(), dtype=tf.resource, name=None)
37958352: TensorSpec(shape=(), dtype=tf.resource, name=None)
37959312: TensorSpec(shape=(), dtype=tf.resource, name=None)
verted and saved as mnist_model.tflite
ails: [{'name': 'serving_default_keras_tensor:0', 'index': 0, 'shape': array([ 1, 28, 28], dtype=int32)},
tails: [{'name': 'StatefulPartitionedCall_1:0', 'index': 9, 'shape': array([ 1, 10], dtype=int32)}]
l/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:
20. Please use the LiteRT interpreter from the ai_edge_litert package.
he [migration guide](https://ai.google.dev/edge/litert/migration)
etails.

:s.warn(_INTERPRETER_DELETION_WARNING)

```

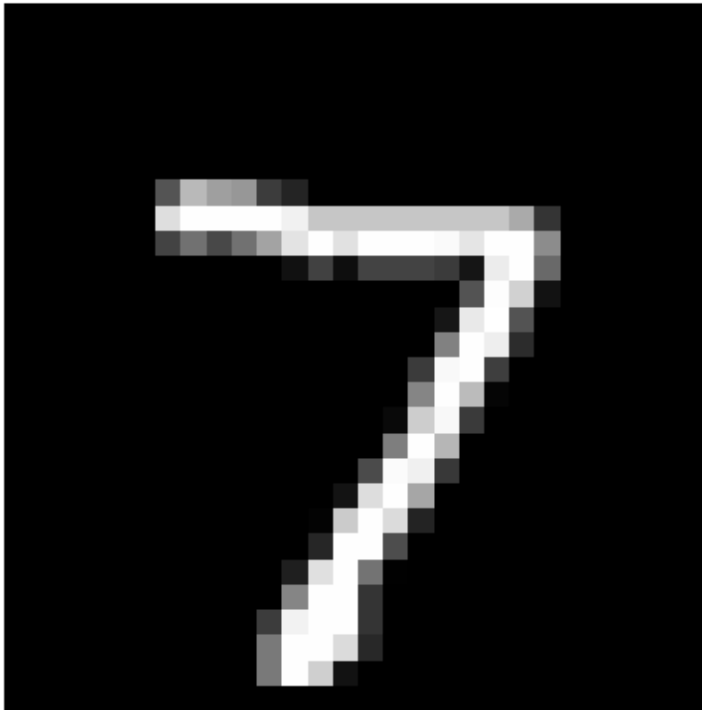
Step 7: Performing Inference Final Step

```

1 # Picking a test image index (change idx to test different ones)
2 idx = 0
3
4 # Preparing input (must match shape and dtype expected by the TFLite model)
5 test_image = x_test[idx] # shape (28, 28)
6 test_image = np.expand_dims(test_image, axis=0) # shape (1, 28, 28)
7 test_image = test_image.astype(np.float32)
8
9 # Setting Up input tensor
10 interpreter.set_tensor(input_details[0]["index"], test_image)
11
12 # Running inference
13 interpreter.invoke()
14
15 # Get output probabilities
16 output_data = interpreter.get_tensor(output_details[0]["index"]) # shape (1, 10)
17 predicted_label = int(np.argmax(output_data, axis=1)[0])
18
19 # Displaying image and prediction
20 plt.imshow(x_test[idx], cmap="gray")
21 plt.title(f"Predicted: {predicted_label} | Actual: {y_test[idx]}")
22 plt.axis("off")
23 plt.show()
24
25 print("Raw probabilities:", output_data)
26

```

Predicted: 7 | Actual: 7



Raw probabilities: $[[3.8996735e-08 \ 5.8953344e-09 \ 2.5966156e-06 \ 3.5066547e-05 \ 4.242946.7248540e-09 \ 2.7025025e-12 \ 9.9995399e-01 \ 1.3900286e-06 \ 6.9316948e-06]]$

a. This Final output proves thst TensorFlow Lite model loaded prfectly. b. Inference ran successfully using the TFLite interpreter c. Finilly The digit was predicted correctly which means the end to end pipeline worked properly.

End Of Work

This extras work on Github is just to save my Project on Github Repos.

```
1 from google.colab import userdata
2 userdata.get('Git-Token')
```

```
'ghp_3i4AnlUnsojJf5lQtbG9TC2PPaEAKH1xcZbf'
```

```
1 !git init
2 !echo "# ITAI-3377-TensorFlow-Keras" >> README.md
3 !git add README.md
4 !git commit -m "AI model-tensorflow-keras"
5 !git branch -M main
6
7 !git remote set-url origin https://joelleyarro03:${github_token}@github.com/joelleyarro03/ITA
8 !git push -u origin main
```

```
ized existing Git repository in /content/.git/
df71] AI model-tensorflow-keras
hanged, 1 insertion(+)
nvalid username or token. Password authentication is not supported for Git operations
```

Authentication failed for '<https://github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git>'

```
1 from google.colab import userdata
2
3 # Retrieve your GitHub Personal Access Token from Colab Secrets.
4 # The secret is named 'Git-Token' based on previous execution.
5 github_token = userdata.get('Git-Token')
6
7 !git init
8 # Set the remote URL using the retrieved token and the correct repository path.
9 !git remote set-url origin https://joelleyarro03:${github_token}@github.com/joelleyarro03/ITA
10
```

Reinitialized existing Git repository in /content/.git/

```
1 !git remote -v
```

```
origin https://joelleyarro03:@github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git
origin https://joelleyarro03:@github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git
```

```
1 from google.colab import userdata
2 import subprocess
3
4 # Retrieve the GitHub token using its assigned name 'Git-Token'.
5 token = userdata.get("Git-Token")
6 assert token, "Git-Token secret not found"
7
8 repo_url = f"https://joelleyarro03:{token}@github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git"
9 subprocess.run(["git", "remote", "set-url", "origin", repo_url], check=True)
10 print("Remote set.")
```

Remote set.

```
1 !git push -u origin main
```

```
Enumerating objects: 48, done.
Counting objects: 100% (48/48), done.
Delta compression using up to 2 threads
Compressing objects: 100% (32/32), done.
Writing objects: 100% (48/48), 3.61 KiB | 527.00 KiB/s, done.
Total 48 (delta 15), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (15/15), done.
remote: error: GH007: Your push would publish a private email address.
remote: You can make your email public or disable this protection by visiting:
remote: https://github.com/settings/emails
To https://github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git
! [remote rejected] main -> main (push declined due to email privacy restrictions)
error: failed to push some refs to 'https://github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git'
```

```
1 !git config --global user.email "joelleyarro03@users.noreply.github.com."
2 !git config --global user.name "joelleyarro03"
```

```
1 !git commit --amend --reset-author --no-edit
2 !git push -u origin main --force
```

```
[main d220fa3] AI model-tensorflow-keras
1 file changed, 1 insertion(+)
Enumerating objects: 48, done.
Counting objects: 100% (48/48), done.
Delta compression using up to 2 threads
Compressing objects: 100% (32/32), done.
Writing objects: 100% (48/48), 3.62 KiB | 463.00 KiB/s, done.
Total 48 (delta 15), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (15/15), done.
To https://github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git
* [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

```
1 !git config --global --list | grep user
2
```

```
user.email=joelleyarro03@users.noreply.github.com.
user.name=joelleyarro03
```

```
1 !git add .
2 !git commit -m "Updated"
3 !git push
4
```

```
[main 3c7f18f] message
24 files changed, 51059 insertions(+)
create mode 100644 .config/.last_opt_in_prompt.yaml
create mode 100644 .config/.last_survey_prompt.yaml
create mode 100644 .config/.last_update_check.json
create mode 100644 .config/active_config
create mode 100644 .config/config_sentinel
create mode 100644 .config/configurations/config_default
create mode 100644 .config/default_configs.db
create mode 100644 .config/gce
create mode 100644 .config/hidden_gcloud_config_universe_descriptor_data_cache_conf:
create mode 100644 .config/logs/2026.01.16/14.23.31.981136.log
create mode 100644 .config/logs/2026.01.16/14.24.03.314209.log
create mode 100644 .config/logs/2026.01.16/14.24.13.071214.log
create mode 100644 .config/logs/2026.01.16/14.24.18.954466.log
create mode 100644 .config/logs/2026.01.16/14.24.28.646070.log
create mode 100644 .config/logs/2026.01.16/14.24.29.392089.log
create mode 100644 mnist_model.h5
create mode 100644 mnist_model.keras
create mode 100644 mnist_model.tflite
create mode 100755 sample_data/README.md
create mode 100755 sample_data/anscombe.json
create mode 100644 sample_data/california_housing_test.csv
create mode 100644 sample_data/california_housing_train.csv
create mode 100644 sample_data/mnist_test.csv
create mode 100644 sample_data/mnist_train_small.csv
Enumerating objects: 32, done.
Counting objects: 100% (32/32), done.
Delta compression using up to 2 threads
Compressing objects: 100% (24/24), done.
```

```
Writing objects: 100% (31/31), 10.85 MiB | 1.60 MiB/s, done.  
Total 31 (delta 5), reused 0 (delta 0), pack-reused 0  
remote: Resolving deltas: 100% (5/5), done.  
To https://github.com/joelleyarro03/ITAI-3377-TensorFlow-Keras.git  
d220fa3..3c7f18f  main -> main
```